

Theory:

1) A function declaration defines a named function using the function keyword. A function expression creates a function and assigns it to a variable. The main difference is that function declarations are hoisted, while function expressions are not available before their initialization.

2) No, function declarations are fully hoisted, so they can be called before they are written in the code. Function expressions assigned to `let` or `const` cannot be accessed before their initialization.

3) An arrow function is a shorter way to write a function using the `=>` syntax.

Eg) `const add = (a, b) => a + b;`

It has shorter syntax than a regular function and does not have its own `this`.

4) No, An arrow function does not have its own `this`. It inherits this from its surrounding lexical scope. Therefore, when an arrow function is used as an object method, this does not automatically refer to that object.

5) A callback function is a function passed as an argument to another function and executed by that function when needed.

Callbacks are commonly used with array methods, events and asynchronous operations.

6) Callbacks allow us to specify what action should be performed on each array ~~for~~ element or what should happen after an operation finishes. For eg) `map()`, `filter()`, and asynchronous operations commonly accept callback functions.

7) A function expression can be either named or anonymous. For eg) a function expression can have an internal name that can be used inside the function itself.

8) Calling a function means executing it using parentheses.
`great();`

Referencing a function means referring to the function without executing it:
`great;`

Therefore `great()` executes the function, while `great` only refers to it.

9) Default parameters provide a default value when an argument is not supplied. They are useful when a function should have a fallback value. For eg), a greeting function can use "Hello" as the default greeting when no greeting is provided.

10) Common ways include:

- 1) Regular function invocation
- 2) Method invocation
- 3) Constructor invoking using `new`
- 4) Invoking `call()`

5. Invoking using `ply()`

6. Invoking of a function returned from `bind()`

Output prediction:

8) hi

10) hi

13) hi

14) 15

15) [2, 4, 6]

16) Called

17) Sam

18) undefined

19) function

20) function

Error finding:

21) No error. o/p: hi

Reason: Function declarations are hoisted, so the function can be called before its declaration.

22) Reference Error.

Reason: `great` is declared with `const` and is accessed before initialization. It is in the Temporal Dead zone.

23) Type Error: Assignment to constant variable.

Reason: A variable declared with `const` cannot be re-assigned.

24) NaN

Reason: b is not provided, so its value is undefined. Though $1 + \text{undefined}$ results in NaN .

25) undefined

Reason: The arrow function does not bind this to obj .

26) Syntax Error.

Reason: A function declaration requires a name. An anonymous function must be used as a function expression or another valid function context.

27) Reference Error.

Reason: `named` is the internal name of the named function expression and is not available as a variable in the outer scope.

28) No error.

Reason: `call back` is a function declaration and is hoisted. The function can therefore be called before its declaration. However, relying heavily on this style can reduce code readability.

29) Syntax Error.

Reason: There is an invalid stray comma after the arrow function expression.

Correct form:

`const arrow = (a, b) => a + b;`

30) Nothing is printed.

Reason: cb only references the callback. It does not execute it. It would need to be

cb();

Coding:

31) function square(number) {
 return number * number;
}

32) const square = function(number) {
 return number * number;
}

33) const square = (number) => number * number;

34) function great(name, prefix = "Hello") {
 return `\${prefix}, \${name}!`;
}

console.log(great("sam"));

O/P: Hello, sam

35) function processTask(callback) {

 console.log("Doing some work");
 callback();

}

processTask(() => {

 console.log("Task completed");
});

36) const number = [1, 2, 3, 4]

const doubled = numbers.map(number => number * 2);
console.log(doubled);

O/P: [2, 4, 6, 8]

```

37) const person = {
    name: "Sam",
    greet: function () {
        console.log(this.name);
    }
};
person.greet();

```

O/P: Sam

```

38) const person = {
    name: "Sam",
    greet: () => {
        console.log(this.name);
    }
};
person.greet();

```

O/P: undefined.

Reason: The arrow function does not have its own this, so this does not refer to person.

```

39) function createFunction (callback) {
    return function () {
        callback();
    };
}
const newFunction = createFunction(() => {
    console.log("Hello");
});
newFunction();

```

40) const greet = function sayHello() {

// sayHello is available only inside this function
console.log("Hello");

};

greet();

The name ~~say~~ sayHello is the internal name of the function expression and is not available in the surrounding scope.

Real World Scenarios;

41) Use a regular function because it gets its this value from how it is called. When called as an object method, this refers to the object. An arrow function does not have its own this.

42) Use map() with a callback because map() transforms each element and returns a new array. For eg) conceptually
Price $\rightarrow$ price + tax.

The call back performs the tax calculation for each price.

43) The validation function can receive a callback that is executed only when a validation succeeds. Therefore
validate $\rightarrow$ if valid $\rightarrow$ execute success callback.

If validation fails, the success callback is not executed.

44) Because an arrow function does not have its own this. It inherits this from its surrounding scope.

Therefore, this may not refer to the class instance or object as ~~expected~~ expected, which can cause properties

accessed through this to become undefined.

45) Use default parameters. The greeting parameter can have "Hello" as its default value when the caller does not provide one.

46) It is called a callback function or callback parameter. A function is passed to another function so that the receiving function can execute it when required.

47) Arrow functions provide short and concise syntax, making small callbacks easier to read and write. For example, they work very cleanly with methods such as `map()` and `filter()`.

48) A common cause is expecting an arrow function to refer to an object. Arrow functions do not have their own `this`; they inherit it from the surrounding scope. Therefore, `this` property may produce undefined.

49) Either can be used. This choice depends on the coding style and requirement. A function declaration is hoisted, so it can be called before its declaration. An arrow function assigned to a `const` cannot be accessed before initialization. Therefore, hoisting can influence how the code is organized but it does not make one option universally better.

50) This pattern is common because a payment operation can have different outcomes.

- Success callback - handles successful payment.
- Failure callback - handles failed payment or errors.

It is commonly used in asynchronous. Javascript because the result of an operation may be available later, and different actions need to be performed depending on the result.